

## Prueba de concepto de arquitectura

Entrega 4 (parcial) — Hogar de los Alpes

MISW4406 · Diseño y construcción de soluciones no monolíticas · 2026-14

- Luis Fernando Ruiz
- Sara Catalina González
- Germán Jiménez
- Jesús Alvernia

Septiembre de 2026

### Enlaces

Repositorio público: <https://github.com/MISW-4406-No-monoliticas-entregas/hogar-alpes>

Despliegue en Google Compute Engine, VM hogar-alpes-poc (e2-standard-4, us-central1-a), IP pública 35.254.132.236:

| Componente                      | Dirección                                                           |
|---------------------------------|---------------------------------------------------------------------|
| S9 Integraciones Partners       | <a href="http://35.254.132.236:8001">http://35.254.132.236:8001</a> |
| S2 Trabajos Siniestros          | <a href="http://35.254.132.236:8000">http://35.254.132.236:8000</a> |
| S10 Reglas de Partner           | <a href="http://35.254.132.236:8002">http://35.254.132.236:8002</a> |
| S7 Matching de Proveedores      | <a href="http://35.254.132.236:8003">http://35.254.132.236:8003</a> |
| Administración de Apache Pulsar | <a href="http://35.254.132.236:8080">http://35.254.132.236:8080</a> |

Prueba para verificar el sistema en la nube. El primer comando entrega un siniestro en el formato propio del partner; el segundo lo consulta ya procesado por S2, lo que demuestra que el mensaje recorrió el broker:

```
curl -X POST http://35.254.132.236:8001/partners/seguros-alpes/siniestros \
  -H 'Content-Type: application/json' \
  -d '{"numeroReclamo":"DEMO-1","poliza":"POL-1","montoEstimado":500000,
    "moneda":"COP","direccion":{"calle":"Cra 7","ciudad":"Bogota","pais":"CO"}}'
```

```
curl http://35.254.132.236:8000/partners/seguros-alpes/siniestros
```

En la máquina corre el cluster de Pulsar, las cuatro bases de datos y los cuatro servicios. Se apaga fuera de las sesiones de prueba para no consumir créditos; si no responde, el sistema completo se levanta en local con un solo comando, según la sección de verificación de este documento.

## Resumen de la solución

La prueba de concepto implementa los cuatro microservicios que participan en la transacción larga atender un siniestro B2B2C, que representa el 70 % del volumen del negocio. Los servicios se comunican exclusivamente por comandos y eventos sobre Apache Pulsar: no existe ningún llamado HTTP ni gRPC entre ellos. HTTP se usa solamente para las consultas de lectura de cada servicio y para la entrada externa de los partners, que son sistemas de terceros y no servicios de esta arquitectura.

El flujo es el siguiente. El partner envía un siniestro en su propio formato a S9 Integraciones, que actúa como capa anticorrupción: lo traduce al modelo canónico, controla que no llegue dos veces y publica el comando en comandos.siniestros. S2 Trabajos Siniestros consume ese comando, registra los eventos en su almacén y publica los eventos de integración en eventos.siniestros. S10 Reglas de Partner y S7 Matching de Proveedores consumen sus propios tópicos de comandos y publican sus eventos, y además están suscritos a los eventos de siniestros.

En esta entrega parcial los servicios se oyen entre sí por los tópicos pero todavía no completan la transacción: la saga que los coordina y el BFF corresponden a la Entrega 5. Lo que esta entrega deja listo es la infraestructura que hace demostrables los tres escenarios de calidad.

| Servicio                   | Consume                               | Publica                               | Datos                                                               |
|----------------------------|---------------------------------------|---------------------------------------|---------------------------------------------------------------------|
| S9 Integraciones Partners  | HTTP del partner                      | comandos.siniestros, eventos.partners | CRUD, una tabla de sincronizaciones con control de idempotencia     |
| S2 Trabajos Siniestros     | comandos.siniestros                   | eventos.siniestros                    | Event Sourcing con CQRS: almacén de eventos y proyección de lectura |
| S10 Reglas de Partner      | comandos.reglas, eventos.siniestros   | eventos.reglas                        | CRUD, reglas por partner y registro de validaciones                 |
| S7 Matching de Proveedores | comandos.matching, eventos.siniestros | eventos.matching                      | CRUD, proveedores habilitados y asignaciones                        |

Cada servicio tiene su propia instancia de PostgreSQL y ninguno conoce la base de otro. Los tópicos comandos.siniestros y eventos.siniestros están particionados en cuatro, porque son los que reciben el pico de carga; la llave de partición es el identificador del siniestro, lo que preserva el orden dentro de cada siniestro y permite paralelismo entre siniestros distintos.

## Escenarios de calidad

Se valida un escenario por cada atributo de calidad priorizado en las entregas anteriores. La ejecución medida corresponde a la Entrega 5; esta entrega deja la infraestructura que los hace posibles.

**Escalabilidad.** Un evento climático genera un pico de cuatro veces la carga de siniestros de partners, sostenido durante cuarenta y ocho horas. El sistema lo absorbe mediante tópicos particionados y consumidores con suscripción compartida por llave, que escalan agregando réplicas sin modificar código, y un almacén de eventos que solo añade registros. La medida objetivo es aceptación con percentil 99 menor o igual a dos segundos, sin pérdida de mensajes y con crecimiento del rendimiento al añadir réplicas.

**Modificabilidad.** Se publica la versión dos de un esquema de evento y ningún consumidor requiere redesplicue. Se apoya en un campo nuevo con valor por defecto y en la política de compatibilidad hacia atrás del registro de esquemas. La medida objetivo es cero consumidores redesplicados, con los consumidores de la versión uno leyendo sin error los mensajes de la versión dos.

**Disponibilidad.** Cae una réplica de S2 Siniestros mientras hay carga activa y no se pierde ningún siniestro. Se apoya en la suscripción compartida por llave, en confirmar el mensaje solo después de que la transacción de base de datos se consolidó, y en el control de idempotencia por identificador de mensaje: Pulsar reentrega a las réplicas vivas todo lo que no fue confirmado. La medida objetivo es cero siniestros perdidos y recuperación en menos de treinta segundos.

Los tres son relevantes para el negocio y no triviales: el primero responde a las cifras del caso, veinticinco millones de peticiones diarias que crecerán de cuatro a cinco veces y doce mil trabajos diarios que llegarán a treinta y seis mil; el segundo, a la evolución continua de más de treinta partners con contratos distintos; el tercero, al costo contractual de incumplir los acuerdos de nivel de servicio del canal B2B2C.

## Decisiones de arquitectura

### Tipo de eventos

Los eventos publicados en el broker son eventos de integración **con carga de estado**: transportan los datos del agregado y no solo su identificador. La razón es directa: al estar prohibidos los llamados síncronos entre servicios, un consumidor que recibiera únicamente un identificador no tendría forma de consultar el resto. Llevar el estado permite además que cada servicio mantenga su propio modelo de lectura y absorba la carga localmente, lo que favorece escalabilidad y disponibilidad. El costo aceptado es el acoplamiento al esquema, que se mitiga con el versionamiento descrito enseguida.

Conviene distinguir dos niveles: los eventos de dominio internos de S2, entre el módulo de escritura y el de seguimiento, son eventos en memoria despachados por el mediador del seedwork; los que salen por Pulsar son eventos de integración entre servicios.

## Esquemas y su evolución

Se eligió **Avro** porque el cliente de Python de Pulsar lo soporta de forma nativa y el registro de esquemas viene integrado en el propio broker, sin componentes adicionales que desplegar y mantener; porque la evolución mediante valores por defecto es el mecanismo estándar y es lo que hace demostrable el escenario de modificabilidad; y porque es la tecnología usada en el curso. Protobuf se descartó porque su ventaja principal, gRPC y tipado en múltiples lenguajes, no aplica aquí: no hay gRPC y todo el sistema es Python. JSON plano se descartó por no ofrecer contrato ni evolución controlada.

Todo mensaje viaja dentro de un sobre común con identificador, marca de tiempo, versión de especificación, tipo y contenido. El campo de tipo permite que varios eventos compartan un mismo tópico y que el consumidor despache cada mensaje al manejador correcto.

La política del namespace es de compatibilidad hacia atrás. Sobre esa base, los cambios aditivos, es decir agregar un campo con valor por defecto, se publican como una versión nueva del esquema en el mismo tópico, y los consumidores antiguos siguen funcionando sin redespigüe. Los cambios incompatibles, como eliminar o renombrar un campo, no se hacen sobre el mismo flujo: se crea un tópico con sufijo de versión, se publica en ambos durante la transición y los consumidores migran a su propio ritmo. Cada servicio es dueño de los esquemas de los eventos que publica y de los comandos que consume; el otro lado copia la definición en su propio módulo, porque importar código entre servicios reintroduciría acoplamiento en tiempo de compilación.

## Topología de datos

Se implementó una topología **descentralizada**: cada servicio tiene su propia instancia de PostgreSQL con credenciales propias, ningún servicio conoce el esquema de otro, y los modelos de lectura que un servicio necesita se construyen a partir de eventos y no leyendo la base ajena.

La justificación es la traducción directa de los contextos acotados definidos en la primera entrega, y sobre todo la evidencia del sistema actual: la base compartida fue el anti-patrón documentado del monolito, con equipos bloqueados entre sí y despliegues de tres a cuatro horas, de modo que compartir tablas entre servicios sería reintroducir ese problema con una red de por medio. La decisión favorece modificabilidad, porque cada equipo evoluciona su esquema sin coordinar con los demás; escalabilidad, porque cada base crece según la carga de su servicio; y disponibilidad, porque la caída de una base no arrastra a las otras. El costo aceptado es la consistencia eventual entre servicios y cierta duplicación de datos de referencia. Que S2 tenga dos tablas en su propia base, el almacén de eventos y la proyección, no contradice la descentralización: es un único dueño con separación entre escritura y lectura.

## Patrón de almacenamiento

La decisión es híbrida y se tomó servicio por servicio. **S2 usa Event Sourcing con CQRS** porque es el agregado con más transiciones de estado y con consecuencias contractuales, acuerdos de nivel de servicio y disputas con partners, de modo que el negocio necesita

auditoría completa de qué ocurrió y cuándo; reconstruir el agregado a partir de sus eventos la entrega de forma natural. Es además el servicio que recibe el pico de carga, y un almacén que solo añade registros, sin actualizaciones ni bloqueos, escala mejor en escritura. La proyección de lectura se mantiene por suscripción a los eventos de dominio y las consultas leen únicamente de ella.

**Los otros tres servicios usan CRUD** porque sus agregados son configuración o registros con poca historia, reglas de un partner, proveedores habilitados y control de idempotencia, donde Event Sourcing añadiría complejidad sin beneficio. Tener ambos patrones conviviendo demuestra además que cada equipo decide el suyo, que es una de las libertades que justifica la separación en servicios.

### **Broker y plataforma de despliegue**

Para desarrollo, el archivo de composición principal levanta Pulsar en modo autónomo junto con las cuatro bases y los servicios. Para el despliegue se usa un **cluster real**: ZooKeeper, dos bookies y dos brokers con factor de replicación dos. Esa topología permite, en la Entrega 5, retirar un broker y verificar que el sistema sigue aceptando siniestros, y escalar réplicas de S2 para el escenario de carga. Un contenedor de inicialización crea el tenant, el namespace, los tópicos particionados, la política de compatibilidad y la retención, de modo que no hay pasos manuales.

Se desplegó en una máquina virtual de **Google Compute Engine**. Se descartó Cloud Run porque los consumidores de Pulsar son procesos permanentemente activos y Cloud Run escala a cero, lo que rompe las suscripciones de larga vida. Se descartó Kubernetes porque el costo de operarlo dentro del tiempo disponible era el mayor riesgo del plan, aunque es el destino natural en producción, con Pulsar distribuido en varias zonas. El firewall expone únicamente los puertos de las APIs y el de administración de Pulsar; los puertos binarios del broker no se exponen, porque sus únicos clientes son los servicios dentro de la misma máquina. No hay credenciales en el repositorio: los archivos de composición traen valores de desarrollo y las de despliegue se pasan por variables de entorno.

### **Principios de DDD aplicados**

Cada servicio está organizado en capas con las dependencias apuntando siempre hacia el dominio: la API y la infraestructura dependen del dominio y nunca al revés. El dominio no importa Flask, SQLAlchemy ni el cliente de Pulsar. Las entradas, la API y los consumidores del broker, y las salidas, repositorios y despachadores, son adaptadores sobre puertos declarados en el dominio y la aplicación.

Cada servicio contiene sus agregados con raíz, Siniestro, Sincronizacion, ReglaDePartner y Asignacion, con objetos valor inmutables, fábricas, reglas de negocio explícitas y repositorios declarados como interfaz en el dominio e implementados en infraestructura. Las referencias entre agregados son siempre por identidad, de modo que ninguna transacción cruce fronteras de agregado ni de servicio. La separación entre comandos y consultas es explícita en todos los servicios, con manejadores propios.

El seedwork con las clases base está copiado en cada servicio y no compartido como librería. Es una decisión deliberada: una librería común que evoluciona obliga a redespargar los cuatro servicios a la vez, que es precisamente el núcleo compartido disfrazado que causó el problema del monolito. La copia cuesta poco y mantiene cada servicio desplegable de forma independiente.

### **Cómo levantar y verificar el sistema completo**

Un solo comando levanta el broker, las cuatro bases y los cuatro servicios. Los tópicos, el tenant, el namespace y las políticas los crea el contenedor de inicialización, sin pasos manuales:

```
git clone https://github.com/MISW-4406-No-monoliticas-entregas/hogar-alpes.git
cd hogar-alpes
docker compose up --build
```

Quedan disponibles S9 Integraciones en el puerto 8001, S2 Siniestros en el 8000, S10 Reglas en el 8002, S7 Matching en el 8003, y la administración de Pulsar en el 8080.

**Flujo principal, del partner al siniestro.** El partner envía el siniestro en su propio formato; S9 lo traduce, publica el comando y S2 lo procesa hasta la proyección consultable. Un segundo envío con el mismo número de reclamo responde 409, lo que muestra el control de idempotencia.

```
curl -X POST http://localhost:8001/partners/seguros-alpes/siniestros \
-H 'Content-Type: application/json' \
-d '{"numeroReclamo":"SA-1001","poliza":"POL-123","montoEstimado":500000,
  "moneda":"COP","direccion":{"calle":"Cra 7 # 1-2","ciudad":"Bogota","pais":"CO"}}'
```

```
curl http://localhost:8000/partners/seguros-alpes/siniestros
curl http://localhost:8000/siniestros/<id_siniestro>
```

**Ciclo de vida por el tópico de comandos y reconstrucción de la proyección.** Los comandos restantes del agregado se publican en comandos.siniestros con el script incluido en S2; después se borra la proyección a propósito y se reconstruye desde el almacén de eventos, que es la demostración de Event Sourcing.

```
docker compose exec siniestros python scripts/publicar_comando.py \
  asignar <id_siniestro> prov-99
docker compose exec siniestros python scripts/publicar_comando.py validar <id_siniestro>
```

```
docker compose exec postgres-siniestros psql -U siniestros \
-c "SELECT siniestro_id, tipo, version FROM eventos_siniestro ORDER BY version;"
docker compose exec postgres-siniestros psql -U siniestros -c "DELETE FROM
estado_siniestro;"
curl -X POST http://localhost:8000/admin/proyecciones/estado-siniestro/reconstruir
curl http://localhost:8000/siniestros/<id_siniestro>
```

**Reglas de partner y matching de proveedores.** Sus comandos se publican con los scripts de cada servicio, que usan el mismo esquema Avro que consumirá la saga en la Entrega 5. La semilla de S10 permite mostrar aprobación y rechazo; la de S7, asignación y ausencia de proveedor.

```
docker compose exec reglas python tools/publicar_comando.py \
sin-001 seguros-alpes 500000 plomeria bogota-norte
docker compose exec reglas python tools/publicar_comando.py \
sin-002 banco-andes 9000000 plomeria bogota-norte
curl http://localhost:8002/partners/seguros-alpes/validaciones
```

```
docker compose exec matching python scripts/publicar_prueba.py asignar sin-001
plomeria norte
docker compose exec matching python scripts/publicar_prueba.py asignar sin-003
vidrieria norte
curl "http://localhost:8003/proveedores?servicio=plomeria&zona=norte"
```

**Los eventos en el broker.** Cualquier tópico se puede observar directamente, y las estadísticas de administración muestran las particiones, las suscripciones compartidas por llave y los esquemas registrados.

```
docker compose exec pulsar bin/pulsar-client consume \
persistent://hogar-alpes/siniestros-b2b2c/eventos.siniestros -s demo -n 0 -p Earliest
docker compose exec pulsar bin/pulsar-admin topics partitioned-stats \
persistent://hogar-alpes/siniestros-b2b2c/comandos.siniestros
docker compose exec pulsar bin/pulsar-admin schemas get \
persistent://hogar-alpes/siniestros-b2b2c/eventos.siniestros
```

**Pruebas automatizadas.** Cada servicio trae sus pruebas del dominio; las de S2 cubren además el almacén de eventos, con reconstrucción, versiones y conflicto de concurrencia, y la compatibilidad entre los esquemas v1 y v2. Se corren dentro de la carpeta de cada servicio con pytest.

La colección de Postman en `infra/postman/` recorre el flujo completo, y el `README.md` del repositorio documenta cada paso con más detalle.

### Descripción de actividades por miembro

Las contribuciones están registradas en los commits y pull requests del repositorio. Cada servicio se desarrolló en su propia rama y entró a main por pull request revisado por otro integrante.

**Luis Fernando Ruiz.** Reestructuración del repositorio a monorepo con servicios autocontenidos. Infraestructura del broker: archivos de composición para desarrollo y para el cluster, y script de creación de tenant, namespace, tópicos particionados, política de compatibilidad y retención. Servicio de referencia que sirvió de base a los demás. Implementación completa de S9 Integraciones Partners, con el agregado de sincronización, los traductores por partner que actúan como capa anticorrupción, el control de

idempotencia, comando y consultas, esquemas, repositorio y despachador. Integración final: alineación de las copias de esquemas entre servicios y ajuste del cluster para incluir los cuatro servicios. Despliegue en Google Compute Engine y scripts de aprovisionamiento. README de arquitectura y decisiones. Pull requests 3, 4 y 8.

**Sara Catalina González.** Migración de S2 Trabajos Siniestros a Event Sourcing: sustitución de la persistencia CRUD por un almacén de eventos como fuente de verdad, con reconstrucción del agregado a partir de sus eventos y control de concurrencia por versión. Consumidor del tópico de comandos con suscripción compartida por llave, confirmación posterior a la consolidación en base de datos e idempotencia, y despacho por tipo de comando. Comandos de validación y rechazo, comando administrativo de reconstrucción de la proyección, esquema v2 del evento de siniestros y pruebas del almacén de eventos y de compatibilidad de esquemas. Corrección del contrato del comando entre S9 y S2. Pull request 6.

**Jesús Alvernia.** Implementación de S10 Reglas de Partner: agregado de reglas y registro de validaciones sobre modelo CRUD, con datos semilla de tres partners. Consumidor del tópico de comandos que evalúa monto cubierto, servicios y zonas, y publicación de los eventos de aprobación y rechazo. Suscripción al tópico de eventos de siniestros y alineación de su copia del contrato con el dueño. Herramienta para publicar comandos de prueba, consultas de lectura y pruebas del agregado. Revisión y fusión de pull requests de otros integrantes. Pull request 7.

**Germán Jiménez.** Implementación de S7 Matching de Proveedores: agregado de asignación y modelo de lectura de proveedores habilitados con datos semilla. Esquemas de comandos y eventos del servicio, consumidor y despachador, comandos de asignación y liberación, esta última la compensación que usará la saga en la Entrega 5, y consulta de proveedores por zona y servicio. Pruebas del agregado, empaquetado del servicio, integración en el archivo de composición y script de prueba manual documentado. Pull request 5.

### **Alcance de esta entrega**

Conforme al enunciado, la Entrega 4 cubre los puntos uno a seis y el nueve: microservicios con comunicación asíncrona sobre Apache Pulsar, definición y evolución de esquemas, topología de almacenamiento y despliegue. La coordinación de la transacción larga mediante un patrón de sagas, el desarrollo del BFF, la consolidación de los resultados cuantitativos de la experimentación y el refinamiento del mapa de contextos y de los puntos de vista corresponden a la Entrega 5.